



Class and Object

Software Development COS7009–B

Classes and objects

- **class**: A program entity that represents either:
 1. A program / module, or
 2. A template for a new type of objects.
- **object**: An entity that combines state and behavior.
 - **object-oriented programming (OOP)**: Programs that perform their behavior as interactions between objects.



Blueprint analogy

iPod blueprint

state:

current song
volume
battery life

behavior:

power on/off
change station/song
change volume
choose random song



creates

iPod #1

state:

song = "1,000,000 Miles"
volume = 17
battery life = 2.5 hrs

behavior:

power on/off
change station/song
change volume
choose random song



iPod #2

state:

song = "Letting You"
volume = 9
battery life = 3.41 hrs

behavior:

power on/off
change station/song
change volume
choose random song



iPod #3

state:

song = "Discipline"
volume = 24
battery life = 1.8 hrs

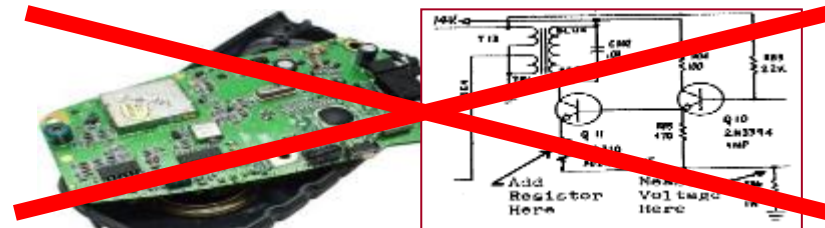
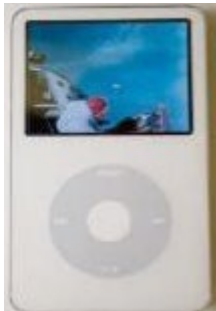
behavior:

power on/off
change station/song
change volume
choose random song



Abstraction

- **abstraction:** A distancing between ideas and details.
 - We can use objects without knowing how they work.
- abstraction in an iPod:
 - You understand its external behavior (buttons, screen).
 - You don't understand its inner details, and you don't need to.





Our task

- In the following slides, we will implement a `Point` class as a way of learning about defining classes.
 - We will define a type of objects named `Point`.
 - Each `Point` object will contain x/y data called **fields**.
 - Each `Point` object will contain behavior called **methods**.
 - **Client programs** will use the `Point` objects.

Point objects (desired)

```
p1 = Point(5, -2)
p2 = Point()           // origin, (0, 0)
```

- Data in each `Point` object:

Field name	Description
x	the point's x-coordinate
y	the point's y-coordinate

- Methods in each `Point` object:

Method name	Description
<code>setLocation(x, y)</code>	sets the point's x and y to the given values
<code>translate(dx, dy)</code>	adjusts the point's x and y by the given amounts
<code>distance(p)</code>	how far away the point is from point <i>p</i>

Point class as blueprint

Point class

```
state:  
int x, y  
behavior:  
setLocation(int x, int y)  
translate(int dx, int dy)  
distance(Point p)
```

Point object #1

```
state:  
x = 5, y = -2  
behavior:  
setLocation(int x, int y)  
translate(int dx, int dy)  
distance(Point p)
```

Point object #2

```
state:  
x = -245, y = 1897  
behavior:  
setLocation(int x, int y)  
translate(int dx, int dy)  
distance(Point p)
```

Point object #3

```
state:  
x = 18, y = 42  
behavior:  
setLocation(int x, int y)  
translate(int dx, int dy)  
distance(Point p)
```

- The class (blueprint) will describe how to create objects.
- Each object will contain its own data and methods.



Creating Classes

```
class Point:
    def setLocation(self, newX, newY) :
        self.x = newX
        self.y = newY
```

- Save this code into a file.
- The above code creates a new type named `Point`.
 - Each `Point` object contains two pieces of data:
 - One named `x`, and
 - One named `y`.
 - **`x` and `y` are fields**: A variable inside an object that is part of its state.

Accessing fields

- Other classes can access/modify an object's fields.

- access: **variable.field**
- modify: **variable.field = value**

- Example:

```
p1 = Point()
p2 = Point()
print('the x-coord is ' + p1.x)    // access
p2.y = 13                          // modify
```

A class and its client

- `Point` is not, by itself, a runnable program.
 - A class can be used by **client** programs.

```
class Point:
    def setLocation(self, newX, newY):
        self.x = newX
        self.y = newY
```

```
p1 = Point()
```

```
p1.setLocation(1,3)
```

```
p1.x = 7
```

```
p1.y = 2
```

(class of objects)





Instance methods

- **instance method (or object method)**: Exists inside each object of a class and gives behavior to each object.

```
def name (self, parameters) :  
    statements
```

Example:

```
def shout (self) :  
    print ("HELLO THERE!")
```

Kinds of methods

- **accessor:** A method that lets clients examine object state.
 - Examples: `distance`, `distanceFromOrigin`
 - has a return statement
- **mutator:** A method that modifies an object's state.
 - Examples: `setLocation`, `translate`

Mutator method questions

- Write a method `setLocation` that changes a `Point`'s location to the (x, y) values passed.
- Write a method `translate` that changes a `Point`'s location by a given dx, dy amount.
 - Modify the `Point` and client code to use these methods.



Accessor method questions

- Write a method `distance` that computes the distance between a `Point` and another `Point` parameter.

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Use the formula:

- Write a method `distanceFromOrigin` that returns the distance between a `Point` and the origin, (0, 0).
 - Modify the client code to use these methods.

Initializing objects

- Currently it takes 3 lines to create a `Point` and initialize it:

```
p = Point()  
p.x = 3  
p.y = 8           // tedious
```

- We'd rather specify the fields' initial values at the start:

```
p = Point(3, 8)   // better!
```

- We are able to do this with most types of objects in Python



Constructors

- **constructor**: Initializes the state of new objects.

```
def __init__(self, parameters) :  
    statements
```

- no return type is specified;
it implicitly "returns" the new object being created
- If a class has no constructor, Python gives it a *default constructor*



Constructor example

```
class Point:
```

```
    // Constructs a Point at the given x/y location.  
    def __init__(self, initialx, initialy):  
        self.x = initialX  
        self.y = initialY
```

```
    ...
```



Class attributes and Static Methods

- **static:** Part of a class, rather than part of an object.
 - Object classes can have static methods *and fields*.
 - Not copied into each object; shared by all objects of that class.



Static fields

Example:

```
class Point:  
    total = 0
```

- **static field:** Stored in the class instead of each object.
 - A "shared" global field that all objects can access and modify.
 - Like a class constant, except that its value can be changed.

Accessing static fields

- Accessing static fields

```
class Point:
    total = 0
    def __init__(self, initialX, initialY):
        self.x = initialX
        self.y = initialY
        Point.total += 1
```

```
P1=Point(1,2)
P2=Point(3,4)
print(Point.total)
```

Static methods

```
// the same syntax you've already used for  
methods Note here 'self' is not a  
parameter
```

```
@staticmethod  
def name(parameters) :  
    statements
```

- **static method:** Stored in a class, not in an object.
 - Shared by all objects of the class, not replicated.
 - cannot access any particular object's fields.

Example

Class Point:

```
total = 0
```

```
def __init__(self, initialX, initialY):
```

```
    self.x = initialX
```

```
    self.y = initialY
```

```
    Point.total += 1
```

```
@staticmethod
```

```
def status():
```

```
    print('total number of points created', Point.total)
```

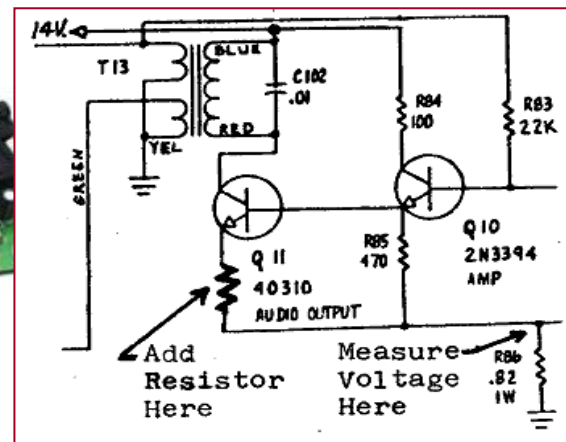
```
P1=Point(1,2)
```

```
P2=Point(3,4)
```

```
print(Point.status())
```

Encapsulation

- **encapsulation:** Hiding implementation details from clients.
 - Encapsulation forces *abstraction*.
 - separates external view (behavior) from internal view (state)
 - protects the integrity of an object's data





Private fields

A field that cannot be accessed from outside the class

`__name` (two underscore characters followed by the field name)

– Examples:

```
class Point:
    def __init__(self, initialX, initialY):
        self.__x = initialX
        self.__y = initialY
```

- Client code won't run if it accesses private fields:

```
P=Point(1,2)
Print(p.x)
```

Accessing private state

```
// A "read-only" access to the x field ("accessor")
def getX(self):
    return self.__x
```

```
// Allows clients to change the x field ("mutator")
def setX(self, newX):
    self.__x = newX
```

- Client code will look more like this:

```
p1=Point(1,2)
print(p1.getX());
p1.setX(14);
```

Private Methods

- Adding two underscores to the method name
- Syntax:

```
def __private_method(self) :  
    statements
```

- Accessing private methods
- Via Public method

```
def public_method(self) :  
    self.__private_method()  
    statements
```

Example

Class Point:

```
def __init__(self, initialX, initialY):  
    self.__x = initialX  
    self.__y = initialY  
def __getX(self):  
    return self.__x  
def __getY(self):  
    return self.__y  
def printCoordinates(self):  
    print(self.__getX(), self.__getY())
```

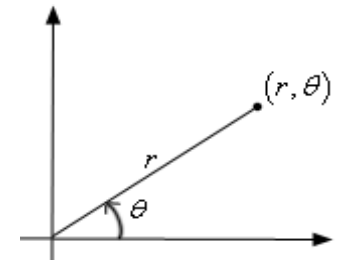
```
P1=Point(1,2)
```

```
P2=Point(3,4)
```

```
print(Point.printCoordinates())
```

Benefits of encapsulation

- Abstraction between object and clients
- Protects object from unwanted access
 - Example: Can't fraudulently increase an `Account`'s balance.
- Can change the class implementation later
 - Example: `Point` could be rewritten in polar coordinates (r, θ) with the same methods.
- Can constrain objects' state (invariants)
 - Example: Only allow `Accounts` with non-negative balance.
 - Example: Only allow `Dates` with a month from 1-12.





Reference

- Python Programming by Michael Dawson, ch8
- <https://legacy.python.org/doc/essays/ppt/>